

AxiomGraph: The Native Cognitive Memory Engine

The Problem with Current GraphRAG Architectures

Most AI engineers building GraphRAG systems today duct-tape a Vector Database (like Pinecone or Milvus) to a Graph Database (like Neo4j) via the application layer. This introduces massive network latency, dual-write consistency issues, and forces the Python application to constantly shuttle IDs and payloads across boundaries to perform hybrid queries.

The AxiomGraph Solution

AxiomGraph is an embedded hybrid Cognitive Memory Engine that *natively fuses* a Graph Database (CSR) with a GPU-Accelerated Vector Database (NVIDIA cuVS) in a single C++ memory space.

It is designed specifically for autonomous AI agents that require both **Semantic Search** and **Logical Reasoning** simultaneously, with zero-copy memory transfers and sub-millisecond latencies.

Key Architectural Advantages

- 1. Zero-Copy Memory Space:** Uses a Struct of Arrays (SoA) and flat 1D memory layouts. Graph topology and semantic vectors share the exact same ID allocator. Data goes straight from the CPU to the GPU for indexing without serialization overhead.
 - 2. Two-Tier Memory Model:**
 - **Hot (Episodic) Memory:** A highly mutable scratchpad where agents can write facts and relationships in real-time without triggering heavy index rebuilds.
 - **Cold (Semantic) Memory:** Calling `db consolidate()` flattens the working memory into a highly compressed CSR (Compressed Sparse Row) graph and builds the static Vector Index (using CPU SIMD or GPU CAGRA).
 - 3. Embedded First:** Runs natively inside the application process via Python bindings (`nanobind`), eliminating network latency completely.
-

Python API Reference

AxiomGraph provides a completely abstracted, Pythonic interface that hides the underlying C++ pointer math.

Initialization

```
from axiomgraph import Memory

# Initializes a new database or loads an existing one from disk.
```

```
# Automatically falls back to CPU SIMD if use_gpu=False or if GPU is un
db = Memory.create(path="./agent_brain", dimensions=768, use_gpu=True)
```

1. Ingestion (Hot Memory)

Agents can ingest facts as nodes (with schemaless JSON properties) and wire them logically.

```
# Ingest vectors and metadata
laptop_id = db.hot.add_node(
    vector=np.array([...], dtype=np.float32),
    label="UltraBook X1",
    type="PRODUCT",
    properties={"price": 1200}
)
category_id = db.hot.add_node(
    vector=np.array([...], dtype=np.float32),
    label="Electronics",
    type="CATEGORY"
)

# Create a logical relationship
db.hot.connect(source=laptop_id, target=category_id, relation="BELONGS_
```

2. Consolidation

Transition Episodic memory into highly optimized Cold Memory. This builds the CSR arrays and the GPU/SIMD vector index.

```
db consolidate()
```

3. GraphRAG Querying

Execute hybrid cognitive search. The engine semantically matches the query vector, then automatically traverses the logical graph topology across multiple hops to assemble the context.

```
# Semantic search for top 2 matches, then traverse 1 hop outward
context = db.query(query_vector, top_k=2, depth=1)

print(context.nodes)
# => [Node(id=0, label='UltraBook X1', type='PRODUCT', properties={'pri

print(context.edges)
# => [Edge(source_id=0, target_id=1, relation='BELONGS_TO', ...)]
```